

Real-Time Incident Management Dashboard

Final Report for the Distributed Systems Course 2025/2026

Alessandro Martini

`alessandro.martini10@studio.unibo.it`

October 6, 2026

This project presents a distributed real-time incident management system designed for collaborative ticket resolution in operational environments. The system enables multiple operators to simultaneously interact with a shared pool of incidents through a web-based Kanban dashboard. The architecture follows a three-tier pattern: a Vue.js 3 single-page application serves as the presentation layer, an Express.js API with Socket.IO handles real-time communication and business logic, while a MongoDB Replica Set ensures persistent strongly-consistent state and Redis provides distributed coordination. Key distributed systems patterns implemented include: (i) Redis-based distributed locking with Lua scripts for atomic claim/release operations, preventing race conditions during ticket assignment; (ii) optimistic locking via document versioning in MongoDB to avoid lost updates during concurrent modifications; (iii) leader election using Redis TTL keys for exclusive execution of background tasks (automatic escalation of stale incidents); (iv) MongoDB Change Streams for reactive, event-driven synchronization across all connected clients without polling. The entire system is containerized and orchestrated via Kubernetes, featuring a 3-node MongoDB StatefulSet for data persistence and failover, Horizontal Pod Autoscaler for the API tier (3–10 replicas), and an Ingress with session affinity.

1 Concept

1.1 Product Type

The project consists of a web application with a real-time graphical user interface (GUI). Specifically, it is a Single-Page Application (SPA) built with Vue.js 3, served as a static web application and interacting with a distributed backend via HTTP and WebSocket protocols. The frontend presents a Kanban-style dashboard that allows multiple operators to visualize, create, claim, and resolve incidents in real time.

1.2 Use Case Description

1.2.1 What the software does

The system implements a **real-time incident management board** (similar to a ticketing or ITSM system) with the following core capabilities:

- **Incident registration:** operators can create new incidents with a title and initial status `open`
- **Ticket claiming:** an operator can *lock* an incident for exclusive processing, preventing other operators from simultaneously working on the same ticket (mutual exclusion)
- **Ticket resolution:** the claiming operator can mark an incident as `closed`, releasing the lock and notifying all connected clients
- **Automatic escalation:** a background task periodically scans for incidents that have remained `open` for more than 5 minutes and automatically escalates them to `escalated` status, signaling SLA breach
- **Real-time synchronization:** all connected clients receive live updates whenever an incident is created, modified, locked, unlocked, or escalated, without manual page refresh
- **User presence tracking:** the system displays the list of currently connected operators

1.3 Why Distribution is Needed

1.3.1 Fault Tolerance

The system must remain operational even if individual components fail. With a single-node deployment, the failure of the API server or the database would render the entire ticketing system unusable—a critical risk in operational environments where incident tracking cannot afford downtime. The Kubernetes architecture addresses this by:

- Running **3 API replicas** minimum, so the loss of one pod does not interrupt service
- Deploying MongoDB as a **3-node Replica Set**, which tolerates the failure of one node while maintaining a primary for writes
- Using Redis for shared state, so any API pod can take over coordination tasks

1.3.2 Scalability

As the number of concurrent operators and incidents grows, a single API instance would become a bottleneck. The system employs a **Horizontal Pod Autoscaler** (HPA) that scales the API tier from 3 to 10 replicas based on CPU utilization, ensuring that:

- HTTP request handling capacity increases with load
- Socket.IO connections are distributed across multiple pods (via the Redis adapter)
- The background escalation task is executed by exactly one leader, regardless of pod count

1.3.3 Resource Sharing

Multiple operators must access and modify the **same shared pool of incidents** concurrently. Without distribution, the system would be limited to a single operator at a time, defeating its collaborative purpose. The distributed architecture enables:

- **Concurrent read access:** all operators see the same live state of the Kanban board
- **Coordinated write access:** distributed locks ensure that only one operator can claim a ticket at a time, while optimistic locking prevents conflicting updates

1.3.4 Geographic Distribution (Potential)

While the current deployment targets a single Kubernetes cluster (e.g., Minikube or a single cloud region), the architecture is designed to support geographic distribution. The MongoDB Replica Set could be extended with nodes in different data centers, and the API tier could be replicated across regions behind a global load balancer, enabling operators in different locations to collaborate on the same incident pool.

2 Requirements Elicitation and Analysis

2.1 Functional Requirements

FR1 Board access. An operator can join the board by registering a username/password pair and, on subsequent visits, logging in with those credentials.

Acceptance criterion: submitting valid credentials on the login form transitions the client from the login screen to the dashboard and establishes a persistent, authenticated WebSocket connection; submitting invalid credentials is rejected with an error and no connection is established.

FR2 Incident creation. An operator can create a new incident by providing a title; the system assigns it a default status of **open**, a creation timestamp, and the creator's identity.

Acceptance criterion: a valid `POST /api/incidents` request returns 201 and the new incident appears as a card in the “Open” column of every connected client without any client issuing a manual refresh.

FR3 Real-time board synchronization. All connected operators must observe incident creations, claims, releases, resolutions, and escalations as they happen.

Acceptance criterion: an action performed by operator A becomes visible on operator B’s board without B reloading the page or polling.

FR4 Ticket claiming (mutual exclusion). An operator can claim an `open` or `escalated` incident for exclusive handling; once claimed, no other operator can claim or resolve it.

Acceptance criterion: of two concurrent claim requests on the same incident, exactly one succeeds (200) and the other is rejected (409, reporting the current owner).

FR5 Explicit claim release. An operator holding a claim can release it without resolving the incident, returning it to the shared pool for other operators.

Acceptance criterion: after a successful release, the incident becomes claimable again immediately, without waiting for the automatic lock expiry.

FR6 Ticket resolution. The operator currently holding the claim on an incident can mark it as `closed`, recording who closed it and releasing the associated lock.

Acceptance criterion: a resolved incident moves to the “Resolved” column on every client and can no longer be claimed or resolved again.

FR7 Automatic SLA escalation. Incidents left in the `open` status beyond a configured threshold (5 minutes) are automatically transitioned to `escalated` without operator intervention.

Acceptance criterion: an incident older than the threshold changes to `escalated` within one execution cycle of the background task (≤ 10 seconds after threshold crossing) and moves to the “Escalated” column on every client.

FR8 Presence awareness. The system displays the set of operators currently connected to the board.

Acceptance criterion: the online-users list reflects a new connection within one broadcast cycle, and a user disappears from the list once their connection is terminated (including ungraceful terminations).

FR9 Filtering and search. An operator can narrow the visible incidents via free-text search on the title, an “escalated only” toggle, and a “mine only” toggle (incidents they created, claimed, or resolved).

Acceptance criterion: activating any filter combination updates only the Kanban columns; the KPI summary counters continue to reflect the true, unfiltered system state.

FR10 Event notifications. Operators receive non-blocking toast notifications for relevant events they did not personally trigger, and for local operation failures.
Acceptance criterion: every failed claim, release, or resolve action produces a visible notification describing the reason for failure; no user-initiated action fails silently.

2.2 Non-Functional Requirements

NFR1 No lost updates. Concurrent modification attempts on the same incident must never silently overwrite one another.

Realized by: optimistic locking via a `version` field, matched on every `PATCH`.

Acceptance criterion: a `PATCH` carrying a stale version is rejected with `409` and leaves the document unmodified.

NFR2 Server-enforced mutual exclusion. The claim mechanism must be enforced by the server, not merely suggested by the client interface.

Acceptance criterion: a resolution request issued by an operator who does not currently hold the incident's lock is rejected with `409`, even when the request bypasses the UI entirely (e.g., sent directly to the API).

NFR3 Fault tolerance of the API tier. The loss of any single API pod must not interrupt service for connected or new operators.

Acceptance criterion: deleting one of the ≥ 3 running `api` pods produces no visible downtime; remaining pods continue serving requests, and `Socket.IO` clients transparently reconnect to a healthy pod.

NFR4 Fault tolerance of the data tier. The loss of any single MongoDB node must not lose committed data, and must restore read/write availability automatically, within a primary election.

Realized by: a 3-node Replica Set with automatic primary election, protected by a `PodDisruptionBudget` (`minAvailable: 2`) that prevents voluntary evictions from breaking quorum.

Acceptance criterion: terminating one of the 3 `mongo` pods triggers automatic recovery (re-election if the primary was lost) without manual intervention, and the API resumes normal operation once the probe reports readiness again.

NFR5 Exactly-once background execution. The automatic escalation task must execute exactly once per cycle across the whole cluster, regardless of how many API pods are running.

Realized by: Redis-based leader election with an atomically-renewed lock (Lua script, preventing the split-brain window of a naive `GET+PEXPIRE`).

Acceptance criterion: with $N \geq 1$ running `api` pods, a given incident is escalated exactly once (no duplicate version increments, no duplicate log entries).

NFR6 Horizontal scalability. The API tier must absorb increased load by adding replicas automatically, without manual intervention.

Realized by: a `HorizontalPodAutoscaler` (3–10 replicas, 80% CPU utilization

target).

Acceptance criterion: sustained CPU load above the threshold results in additional `api` pods being scheduled within the HPA's stabilization window.

NFR7 Reconnection consistency. A client that experiences a temporary disconnection (e.g., during a rolling update of the API tier) must never regress to a state older than what it displayed before disconnecting, and must recover the *whole* observable state of each incident, including who currently holds its claim.

Realized by: a resynchronization-generation counter that discards any hydration response superseded by a later one, and a board snapshot that carries the claim state read from Redis rather than the MongoDB documents alone.

Acceptance criterion: rapid, repeated reconnects never leave the client displaying data fetched by an earlier, superseded hydration; and an operator who reloads the page while holding a claim still sees that claim, and retains the actions it authorizes.

NFR8 Self-healing presence data. The count of online operators must never permanently overcount stale sessions after an ungraceful pod termination.

Realized by: deriving presence from `io.fetchSockets()` (live socket introspection across the Redis adapter) instead of a manually-maintained Redis hash, which was found during development to leak stale entries on unclean shutdowns.

Acceptance criterion: after force-terminating an API pod, the online-user count converges to the correct value without manual state cleanup.

NFR9 UI eventual consistency. A given incident must never be rendered in two mutually-exclusive Kanban columns at once, regardless of the arrival order of independent real-time events describing the same state change.

Acceptance criterion: resolving a claimed incident never produces a transient duplicate card appearing simultaneously in "In Progress" and "Resolved", irrespective of whether the lock-release event or the database change-stream event is delivered to the client first.

2.3 Implementation Requirements

- I1** The backend is implemented in Node.js, using Express for the REST API and Socket.IO for the WebSocket transport. This follows from the design need for a single event-driven runtime capable of hosting both request/response handling and long-lived WebSocket connections in the same process, which simplifies bridging MongoDB Change Stream events onto Socket.IO broadcasts.
- I2** MongoDB is deployed as multi-node Replica Set. A one-member replica set would be sufficient to unlock the Change Streams API, but would not exercise the replication and automatic-failover mechanisms.
- I3** Redis is present as the shared coordination point for all cross-instance mechanisms (Socket.IO Pub/Sub adapter, distributed locks, leader election). This follows di-

rectly from the design decision to keep the API tier stateless and horizontally scalable, which in turn requires an external, shared point of coordination reachable by every replica.

- I4 The system is containerized with Docker and orchestrated with Kubernetes, validated locally on Minikube.
- I5 The frontend is a Vue 3 single-page application using the Composition API.

2.4 Relevant and Non Relevant Distributed System Features

2.4.1 Transparency

Transparency is **partially relevant**. *Location and replication* transparency are provided: the browser never knows which of the 3 to 10 `api` pods serves it, hidden behind the Ingress, nor which `mongo` node is currently primary, hidden behind the replica-set connection string. *Failure* transparency is partial: after a pod failure the client reconnects and resynchronizes on its own instead of surfacing an error. *Concurrency* transparency is deliberately *not* pursued: a claimed incident shows its holder to every operator, because knowing that a colleague is already on it is exactly what prevents duplicated work.

2.4.2 Fault Tolerance, Dependability

Fault tolerance is **relevant**. The system must survive the loss of any one `api` pod (stateless, ≥ 3 replicas) and of any one `mongo` node (3-node Replica Set) without interrupting service, and must never lose or silently corrupt data, which is why concurrent writes use optimistic locking rather than “last write wins”. Recovery is automatic: if the leader fails, its 15-second lock expires and another pod resumes the escalation task within seconds. Redis is the deliberate exception: a single, non-persistent instance holding only coordination state. Its loss costs no incident or credential, but makes the whole board unavailable until it returns, and a restart drops all active claims (section 9).

2.4.3 Scalability

Scalability is **relevant** for the stateless API tier and **out of scope** for the data and coordination tiers. The `api` Deployment scales between 3 and 10 replicas through a `HorizontalPodAutoscaler` on CPU utilization. MongoDB’s 3 nodes exist to tolerate failures, not to add throughput (reads are not routed to secondaries), and Redis is not scaled at all: a conscious trade-off given the project’s scope.

2.4.4 Security, Trust

Security is **deliberately minimized** in favour of the project’s focus on distributed coordination. Operators authenticate with a username and password (bcrypt-hashed, stateless JWT session), but there is a single implicit role, no TLS between components and no encryption at rest.

2.4.5 Resource Sharing

Resource sharing is the **central motivation** for the entire project: multiple SOC operators must concurrently read and modify the *same* shared pool of incidents, which is precisely the scenario that requires distributed coordination rather than a simple single-user application. This single feature justifies two of the three coordination mechanisms implemented: the distributed lock, which serializes exclusive access to an individual incident (claim/release), and optimistic locking, which detects (rather than prevents) conflicting concurrent writes on the shared MongoDB collection.

2.4.6 Openness, Interoperability, Heterogeneity of Components

This feature is **not relevant** to the current scope. The system is a closed application that does not interact with external ticketing or monitoring systems, and does not expose a public, versioned API intended for third-party consumption. The architecture does not preclude such integration in the future (the REST layer could, in principle, be consumed by an external system), but no standardization or interoperability requirement was elicited for this iteration.

2.4.7 Evolvability, Maintainability

This feature is **relevant**. The codebase is split into three independently deployable units (`api/`, `web/`, `k8s/`), and the separation between shallow liveness and deep readiness probes lets the API tier be rolled out without needless restarts or traffic loss. Development also showed that hand-rolled coordination primitives carry a real maintenance cost, since three of them had to be corrected after their first implementation:

- the leadership lock was renewed with a non-atomic `GET+PEXPIRE`, a split-brain risk now closed by an atomic Lua script;
- a manually-maintained “active users” Redis hash leaked stale entries across unclean pod restarts, and was replaced by `io.fetchSockets()`;
- the claim lock existed only as the `incident_locked` broadcast, so it was missing from the snapshot a client hydrates from. A holder who reloaded the page saw their ticket back in “Open” and received a 409 when re-claiming it, locked out until the TTL elapsed, while clients that hydrated at different moments disagreed indefinitely. The fix reads the lock back into the snapshot, makes a re-claim by the holder idempotent and ships the remaining TTL, so that clients expire it on their own.

The last defect is the most instructive, because it was conceptual rather than a race condition and nothing ever failed loudly: any state a client must reconstruct has to be *readable*, not merely *announced*. Coordination primitives require deliberate care to stay correct as the system evolves, not merely to be implemented once.

2.4.8 Performance, Concurrency, Computation/Communication Efficiency, Bandwidth

This feature is **relevant**, though no quantitative target (e.g., a latency SLA) was elicited. For communication efficiency, updates are pushed through MongoDB Change Streams and Socket.IO over WebSocket rather than by polling. The main trade-off is that every **api** pod opens its own Change Stream cursor: each pod then emits only to its local clients, with no duplicate delivery through the Redis adapter, but the primary serves one cursor per pod, a cost that grows linearly as the HPA scales toward 10 replicas.

2.4.9 Economy, Costs

This feature is **not a primary driver** of the design, since the project targets a local Minikube cluster for development and academic validation rather than a production deployment with a defined operating budget.

3 Design

3.1 Architecture

The system follows a three-tier, layered architecture (presentation, application, data), combined with an event-driven / publish-subscribe style for state propagation. It is deliberately *not* split into microservices: the four containers are tiers, not services. **web** only serves static files, **mongo** and **redis** are backing stores shared by every replica, and all domain logic lives in a single **api** service, which hosts the REST endpoints, the WebSocket server, the Change Stream consumer and the escalation task. The WebSocket channel is push-only and is fed directly, inside the same process, by the REST handlers and the Change Stream; splitting them would add inter-service coordination without a benefit at this scale.

Two motivations drive this style, both traceable to section 2:

- **Horizontal scalability:** the **api** tier is stateless *with respect to domain state*, which lets the HPA scale it between 3 and 10 replicas. Any request can be served by any replica, since identity travels in a JWT and domain state lives only in MongoDB (incidents and operator accounts) and Redis (claims, leadership). Each replica still holds disposable *connection* state — its WebSocket sessions and its Change Stream cursor — which a client simply rebuilds by reconnecting elsewhere and rehydrating (section 3.3).
- **Near-real-time propagation:** instead of polling, clients receive events through two channels. Changes to incidents are observed by *every* replica through its own MongoDB Change Stream and pushed to that replica's clients, so the data layer itself is the source of truth for *what changed*; changes to ephemeral state (claims, presence) start on one replica and reach the others through Redis Pub/Sub, which acts as the message broker (section 3.4).

3.2 Infrastructure

The infrastructure is composed of the following components, all containerized and orchestrated by Kubernetes (validated on Minikube):

- **Clients:** operator web browsers, running the Vue.js 3 SPA.
- **Static file server:** an Nginx container (`web` Deployment) serving the compiled Vue.js bundle; stateless, two replicas protected by a `PodDisruptionBudget`.
- **Application servers:** Express.js + Socket.IO processes (`api` Deployment), stateless with respect to domain state (section 3.1), horizontally scaled 3–10 replicas by a `HorizontalPodAutoscaler` on CPU utilization, and protected by a `PodDisruptionBudget` (`maxUnavailable: 1`, a relative bound that adapts to the current replica count).
- **Reverse proxy / load balancer:** the Kubernetes Ingress (Nginx controller, enabled as a Minikube addon), which performs path-based routing (`/` to `web`, `/api/*` and `/socket.io/*` to `api`) and cookie-based sticky sessions across the `api` replicas.
- **Database:** a 3-node MongoDB Replica Set (`StatefulSet`), the durable system of record for both incidents and operator credentials (the `incidents` and `users` collections). Each member owns a `PersistentVolumeClaim` created from the `StatefulSet`'s `volumeClaimTemplates`, so its data survives the recreation of its pod, and a `PodDisruptionBudget` (`minAvailable: 2`) prevents voluntary disruptions from taking away the majority needed to elect a primary.
- **Message broker / coordination store:** a single Redis instance, serving three distinct roles simultaneously — the Socket.IO Pub/Sub adapter (cross-pod Web-Socket broadcast), the distributed-lock store (incident claims), and the leader-election store (background task). It runs as a one-replica Deployment with no persistence and no probes of its own, which makes it the only component without redundancy: since every `api` readiness probe pings Redis, its failure takes *all* `api` replicas out of the Service at once. This single point of failure is a deliberate simplification of the project, not an oversight; replacing it with Redis Sentinel or Redis Cluster is discussed in section 9.

Network placement. In the validated deployment, all components run within a single Kubernetes cluster (a single-node Minikube instance), but each is an independently scheduled Pod communicating exclusively over the cluster's internal network. The architecture therefore does not preclude spreading these components across multiple nodes in a production-grade cluster; this was intentionally left as a deployment-time concern rather than an application-level one.

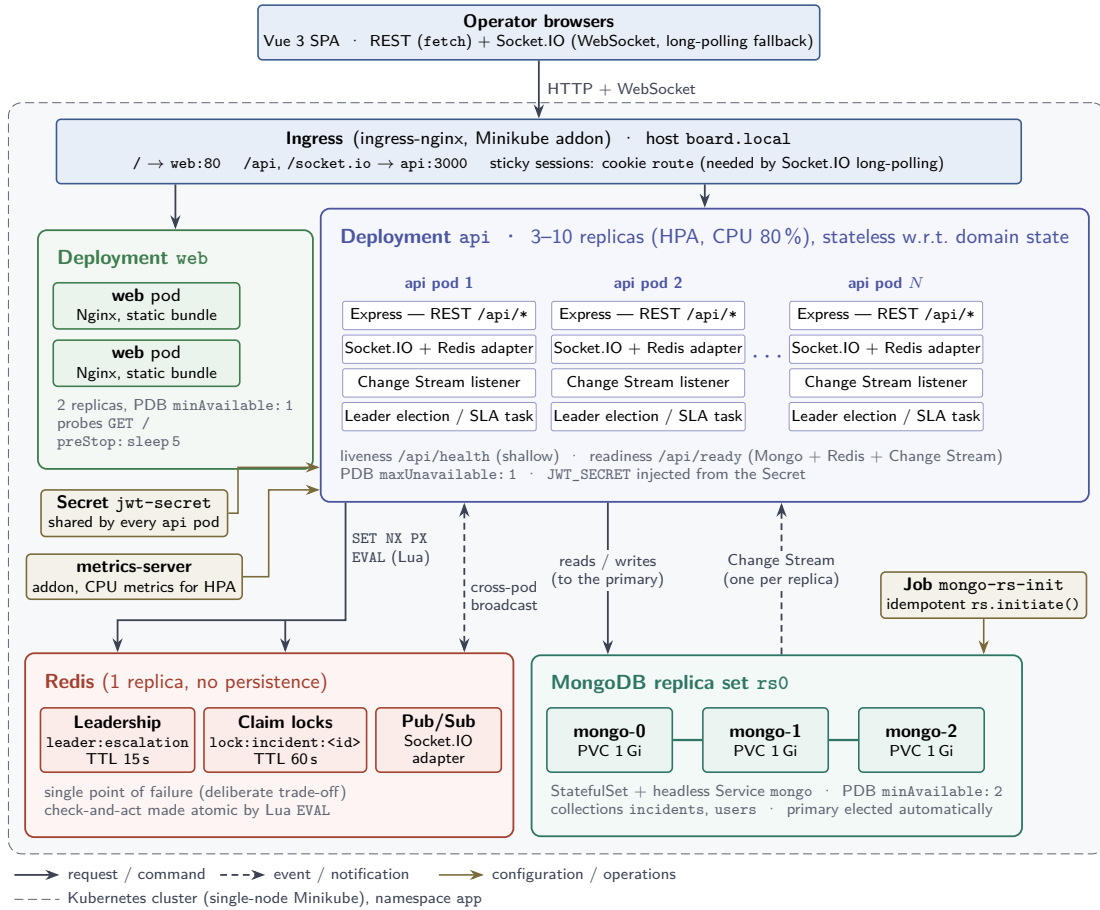


Figure 1: System architecture on the Kubernetes cluster.

Component discovery. Services are located through Kubernetes, with no custom mechanism in the application code:

- **redis** is reached through its `ClusterIP` Service and the cluster DNS (**redis**, i.e. `redis.app.svc.cluster.local`). The **api** and **web** Services, instead, are used by the Ingress only as endpoint lists: it forwards external traffic directly to the pods, which is what allows its affinity cookie to pin a client to one **api** pod.
- MongoDB uses a *headless* Service with the StatefulSet’s stable per-pod names (`mongo-0.mongo, ...`), because replica-set members must be reachable individually rather than load-balanced.
- Externally, the Ingress exposes a single hostname (`board.local`) with path-based routing.

Beyond this, the MongoDB driver discovers the current primary by itself from the seed list in `MONGO_URL`, and the **api** replicas never address each other: they meet only through the shared stores — Redis (adapter channels, `io.fetchSockets()`, the `lock:incident:<id>` and `leader:escalation` keys) and MongoDB (the `version` field checked by optimistic locking) — so the HPA can add or remove replicas without reconfiguring the others.

3.3 Modelling

Domain entities. The core entity is the **Incident** (`title`, `status` $\in \{\text{open}, \text{escalated}, \text{closed}\}$, `createdBy`, `closedBy`, `version`, `updatedAt`). Two further concepts are kept outside it. The **Operator** has persistent credentials (username and bcrypt hash), but its *presence* exists only while its WebSocket connection is open. The **Claim**, the exclusive right to work on an incident, is never written to MongoDB: it is a Redis key with a time-to-live, which is nevertheless read back into the board snapshot at every hydration (section 4.4). Figure 2 shows these entities together with the backend modules that operate on them.

Mapping to infrastructure. This distinction directly reflects each entity’s durability requirement:

- The **Incident** must survive pod restarts, rolling updates, and node failures, and is therefore stored in MongoDB, replicated across the 3-node Replica Set.
- The **Claim** and the **leadership mandate** are, by design, short-lived and self-expiring; persisting them durably would be both unnecessary and actively harmful (a durable lock that outlives its holder’s crashed connection would deadlock the resource). They are therefore stored in Redis, an in-memory store with native TTL support.
- The **Operator**’s *credentials* are persisted in MongoDB, because authentication must outlive any single connection or pod; the operator’s *presence*, by contrast, is not persisted anywhere: it exists only as `socket.data.username` on a live connection, discoverable cluster-wide on demand via `io.fetchSockets()`, and simply ceases to exist the moment the connection closes.

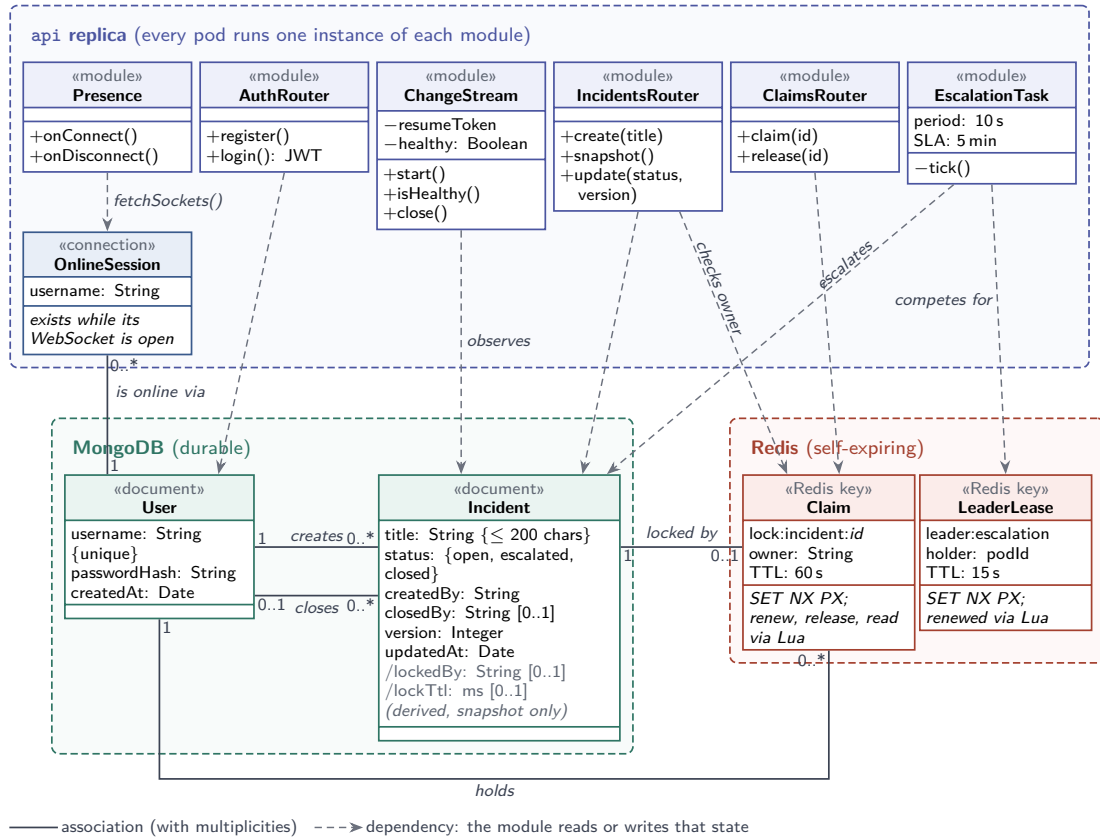


Figure 2: Class diagram.

Domain events. `IncidentCreated`, `IncidentClaimed`, `IncidentReleased`, `IncidentResolved`, `IncidentEscalated`, and `PresenceChanged` are the events that drive the system. They are realized, respectively, as: MongoDB Change Stream notifications for the events that mutate the Incident’s persistent state (created, resolved, escalated), and direct Socket.IO broadcasts for the events that mutate only ephemeral Redis state (claimed, released, presence changed).

Messages. Two distinct message categories are exchanged:

- **commands** (client → server, over REST: `POST /incidents`, `POST/DELETE .../claim`, `PATCH /incidents/:id`), which expect a definite success/failure reply and change system state.
- **events** (server → client, over Socket.IO: `incident_update`, `incident_locked/incident_unlocked`, `users_update`, `resync_required`), which are fire-and-forget notifications with no expected reply. There are no *queries* beyond `GET /api/incidents`, used to hydrate a client’s local state at connection time, after every reconnect, and whenever the server itself asks for a resynchronization via `resync_required` (section 3.7) — after hydration, the client never polls again, relying entirely on the event stream to stay current.

System state. The authoritative state of the system is split across two stores by design: MongoDB holds the durable Incident collection; Redis holds transient coordination state (locks, leadership). Each connected client additionally holds a *derived*, client-local copy of a filtered view of that state (the `incidents` and `onlineUsers` reactive references in the Vue application), which is not authoritative and is kept eventually consistent with the server through the event stream and the reconnect-time resynchronization.

3.4 Interaction

Two interaction patterns coexist, cleanly separated by direction:

- **Client → server: request/response.** Every mutation (create, claim, release, resolve) is a synchronous HTTP call that yields a definite outcome (`2xx` success or `4xx/5xx` failure) before the client considers the action complete.
- **Server → client: publish/subscribe.** All state changes are broadcast to every interested client without any client explicitly requesting them, and without the server tracking who has or hasn’t yet seen a given update. No server-initiated request ever targets one specific client.

Within the publish/subscribe direction, two further sub-patterns are used depending on the origin of the change:

- Changes to **persistent** state (create, resolve, escalate) are captured by MongoDB Change Streams and broadcast via `io.local.emit()` because every `api` pod independently watches the same Change Stream and would otherwise re-broadcast

an event that every other pod has *already* independently received and forwarded to its own clients, causing duplicate delivery.

- Changes to **ephemeral** state not reflected in MongoDB (claim, release, presence) are broadcast via `io.emit()`, which *does* need to cross pod boundaries through the Redis adapter, since only one pod knows the change happened: the one that served the claim or release request, or the one where an operator's WebSocket connected or disconnected. Resolving an incident uses both channels at once: the status change reaches clients through the Change Stream, while the release of its lock is announced with `io.emit()`.

A third case has no originating pod at all: when a claim lock simply *expires* in Redis, no request was served and no pod is aware that anything happened, so no event can be emitted. This category of change is therefore not propagated by a broadcast but *predicted* by each client independently, from the remaining time-to-live the server sends alongside every lock. Overlooking this third case was the root of a defect described in section 2.4, *Evolvability, Maintainability*. Two control messages complete the picture: `resync_required` and the eviction of a degraded pod's clients, both addressed only to that pod's own clients (section 3.7).

Sequence diagrams. The four most significant interactions are shown below, each with two `api` pods so that it is visible which messages cross pod boundaries. Figure 3 covers login, the WebSocket handshake, presence and the hydration of the board. Figure 4 shows a claim with its three possible outcomes. Figure 5 shows a resolution, where the two channels run in parallel: the release of the lock travels through Redis Pub/Sub, the status change through the Change Stream of every pod. Figure 6 shows the SLA escalation run by the leader, which writes to MongoDB and leaves the notification to the Change Streams.

3.5 Behaviour

- **api pods** keep no domain data in memory and react to four stimuli: REST requests, handled synchronously; WebSocket connections and disconnections, which recompute the presence list with `io.fetchSockets()`; events from their own Change Stream, forwarded only to their local sockets; and a self-generated 10-second timer that drives the leader-election attempt and, on the leader, the escalation scan.
- **MongoDB** accepts writes only on the primary, which replicates them to the secondaries through the oplog; if the primary becomes unreachable, the majority elects a new one with no application intervention.
- **Redis** is purely reactive: it executes the commands of the `api` pods (`SET NX PX`, `GET`, `EVAL` of the Lua scripts), relays the adapter's Pub/Sub messages, and expires keys on its own when their TTL elapses.

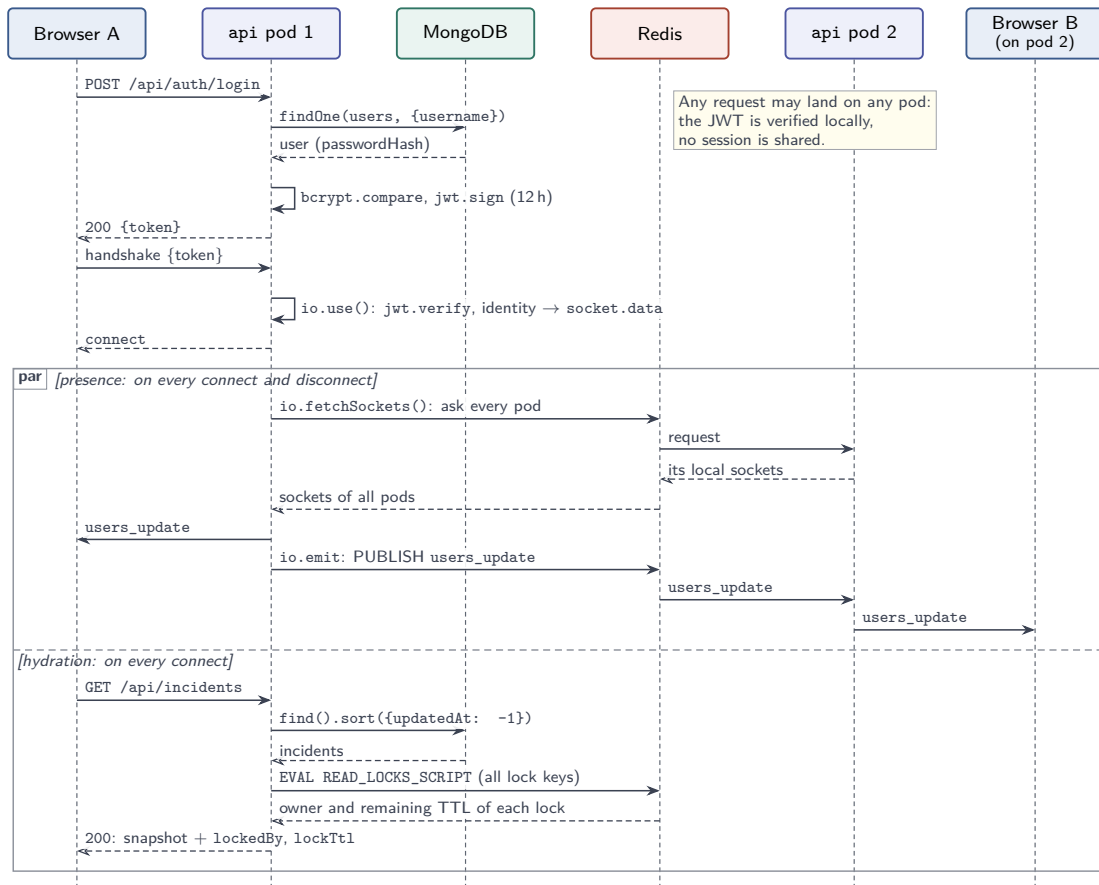


Figure 3: Login, WebSocket handshake, presence and hydration. Presence and hydration both start at every connection and run concurrently.

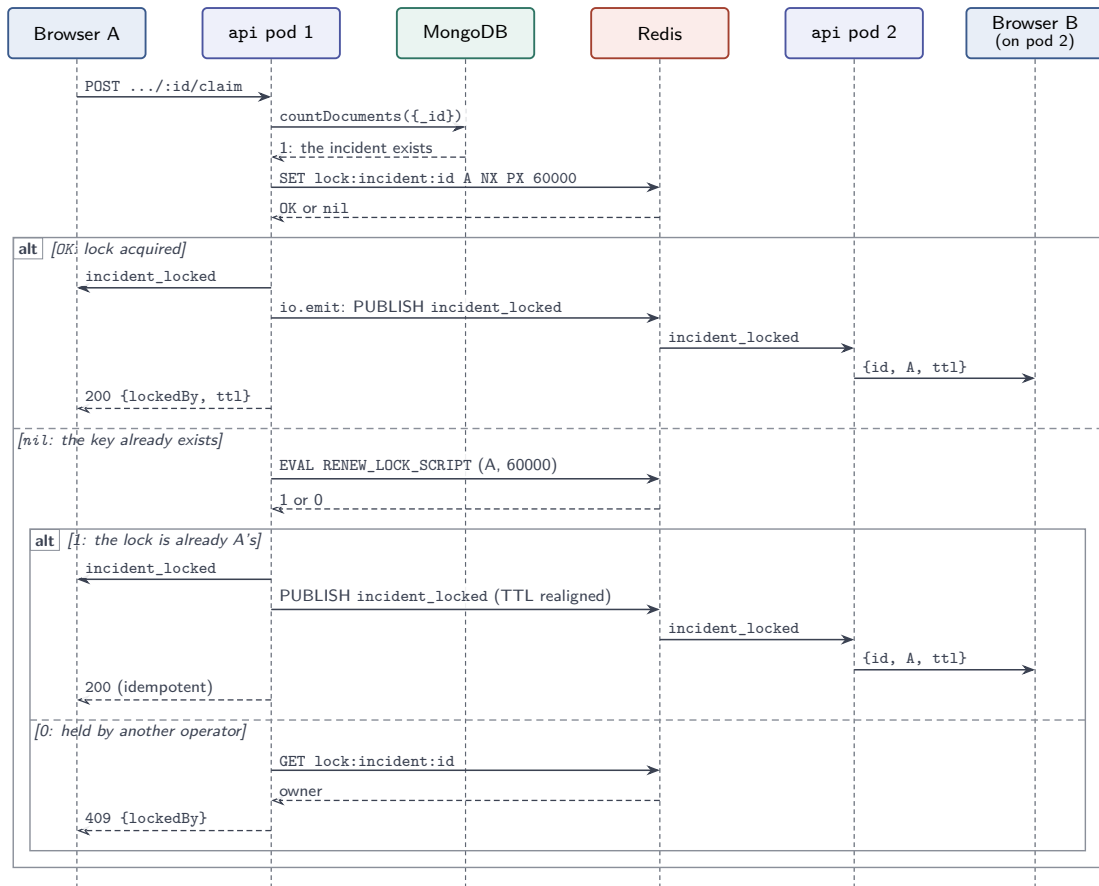


Figure 4: Claiming an incident. The claim is idempotent for the operator who already holds the lock; the 409 is reserved for a conflict with another operator.

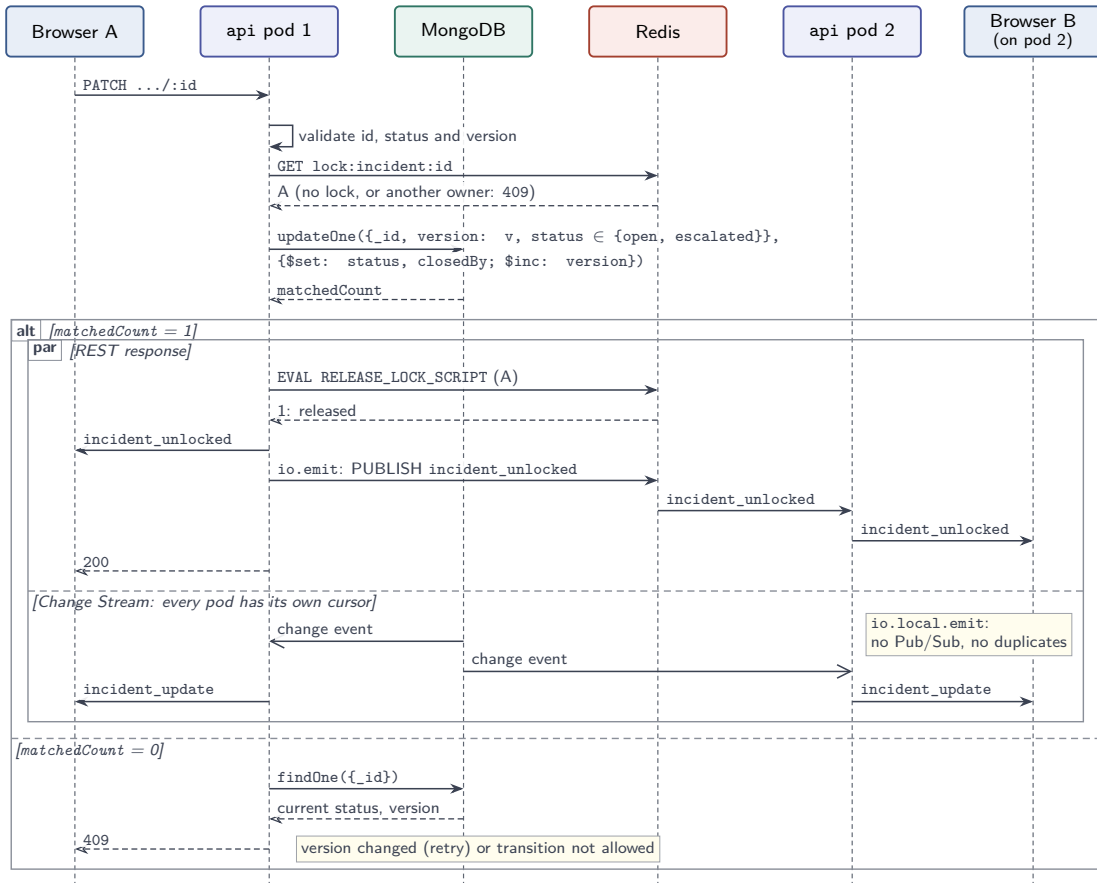


Figure 5: Resolving an incident. The PATCH requires owning the lock, and the update is conditioned on both the version and the allowed source status.

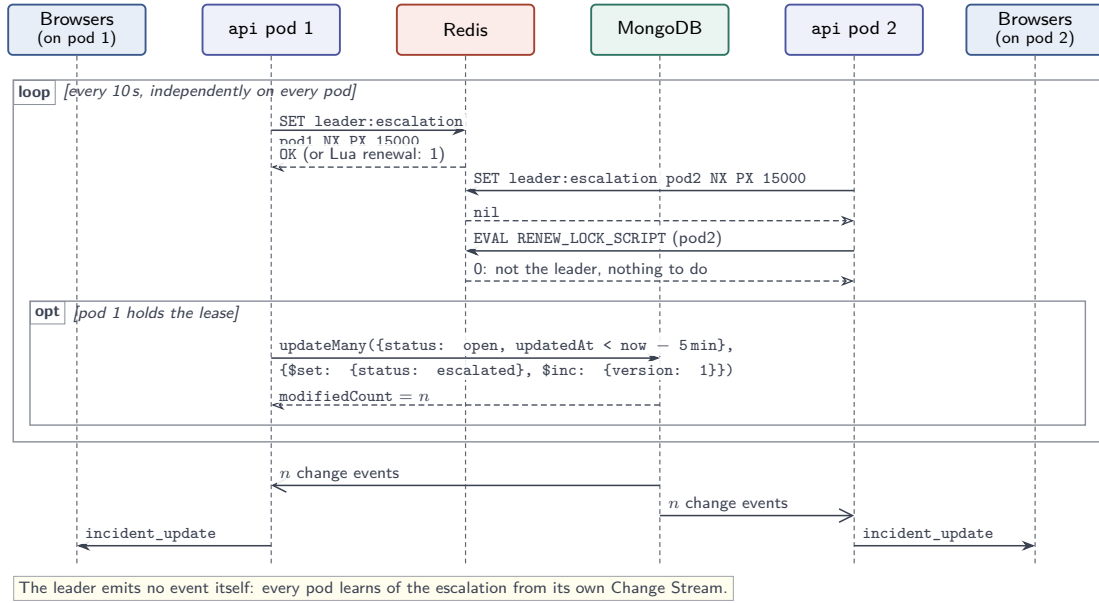


Figure 6: SLA escalation. Every pod competes for the lease at every tick; only the leader writes, and every pod notifies its own clients from its Change Stream.

- **The Vue.js client** never updates its state optimistically: the response to a command only reports its outcome, and the board changes only through server events and snapshots, so it never shows the effect of a command before the server has confirmed it.

State diagrams. Four components have an explicit lifecycle worth modelling. An **incident** (fig. 7) combines its durable status in MongoDB with the ephemeral claim in Redis: the two dimensions are independent, which is why an incident can be escalated while someone is working on it. Each **api pod** has two internal state machines: its **Change Stream** (fig. 8), which decides whether the pod is ready to receive traffic, and its role in the **leader election** (fig. 9). Finally, each **client session** (fig. 10) moves between connecting, hydrating from the REST snapshot and following live events.

Disconnection and cleanup. A client leaves voluntarily (logout closes its socket), through a network failure, or by crashing, which the server notices only when the Socket.IO heartbeat times out. In every case the pod handles the **disconnect** by recomputing presence with `io.fetchSockets()` and broadcasting `users_update`. Nothing else has to be cleaned up, because no per-client state is stored anywhere: presence is derived from the live sockets, and claims are deliberately *not* released on disconnect. A claim belongs to the operator, not to the connection, so a reload or a brief network drop does not cost it, while its 60-second TTL bounds how long an abandoned one can block the incident; logging out likewise leaves held claims to expire.

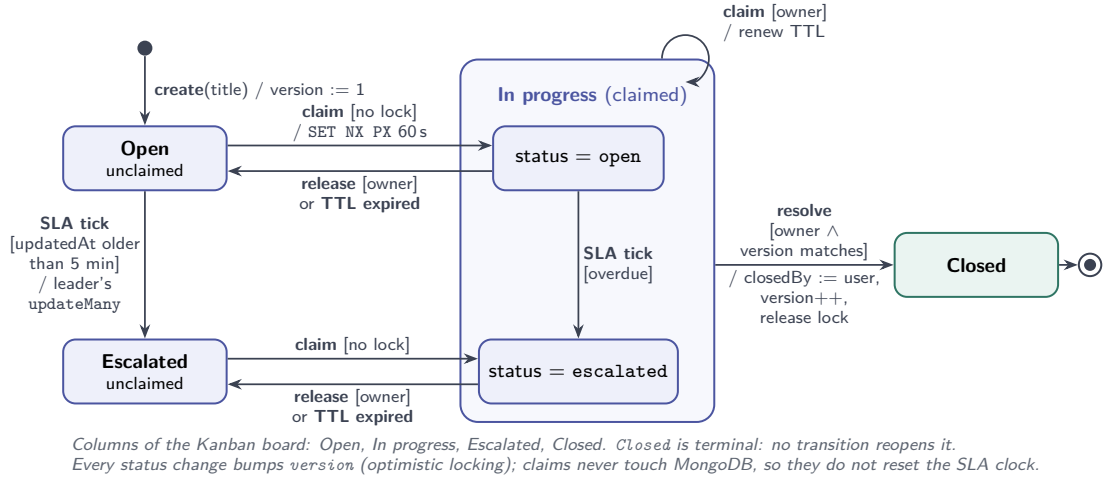
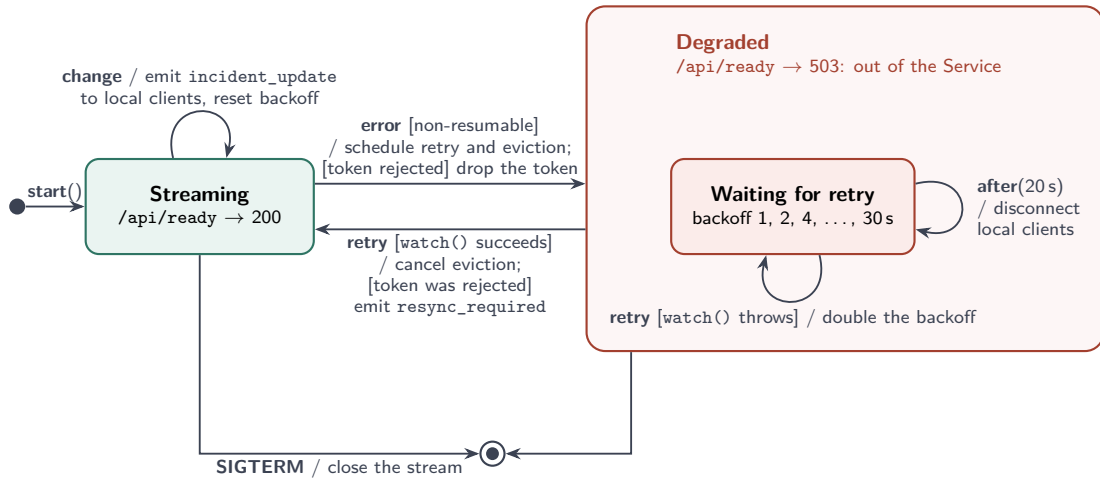
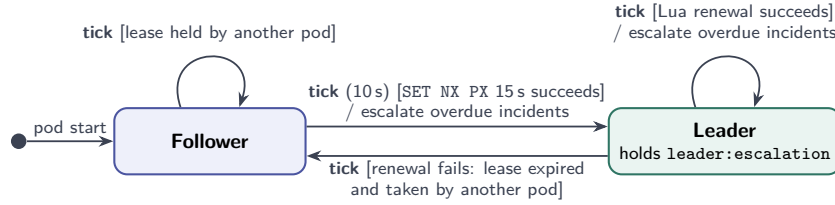


Figure 7: Incident lifecycle. The *In progress* composite state corresponds to the claimed incidents; its two substates keep the status that the incident returns to when the claim is released or expires.



Liveness stays shallow in both states: Kubernetes takes a degraded pod out of the Service but never restarts it. Evicted clients reconnect through the Service, which no longer lists this pod, and rehydrate.

Figure 8: Change Stream of a single *api* pod. Only the *Streaming* state passes the readiness probe; the retry reschedules itself even when `watch()` fails synchronously.



A crashed or terminated leader does not release the lease: another pod takes over within 15 s plus one tick. Without a fencing token, a leader paused for more than 15 s may overlap with its successor; the escalation is idempotent.

Figure 9: Leader election, as seen by a single `api` pod at every 10-second tick.

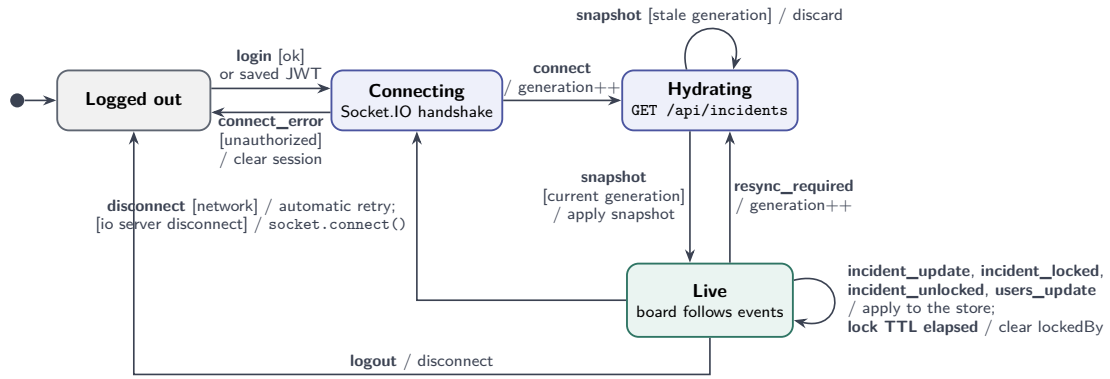


Figure 10: Client session in the browser. The *generation* counter discards snapshots that arrive after a newer resynchronization has started.

3.6 Data and Consistency Issues

What is stored, and where. Incidents are the only data that must persist beyond a single request or connection; they are stored in **MongoDB** as documents. The document model was chosen because incidents are naturally semi-structured records whose schema evolved incrementally over the project’s development (e.g., the **escalated** status and the **closedBy** field were both introduced after the initial design) without requiring a formal migration step. Coordination data (locks, leadership) is stored in **Redis** as plain key-value pairs, chosen because the access pattern is a pure read/write/expire on independent keys with no relational structure, and because lock contention needs sub-millisecond latency that an in-memory store provides naturally.

Query and write patterns. **api** pods are the only components that ever query or write MongoDB and Redis; the client never talks to either data store directly, always mediating through the REST/Socket.IO layer. Concurrent *reads* are unrestricted and require no coordination (both MongoDB and Redis handle concurrent reads natively). Concurrent *writes* are expected and must be explicitly coordinated: this is precisely the motivation behind the two concurrency-control mechanisms of section 2.2 — optimistic locking detects conflicting concurrent writes to the *same* Incident document, and the distributed lock prevents two operators from concurrently claiming the same Incident in the first place.

Shared data. The Incidents collection is, by design, shared read/write state across every **api** replica — this sharing is the entire point of keeping the application tier stateless and horizontally scalable. The same is true of the Redis keyspace: coordination only works if every replica observes the same lock and leadership state, which is why a single shared Redis instance (rather than one per pod) is a hard requirement, not an implementation convenience.

Consistency model. The system provides **strong consistency** for writes to the primary (a successful write is immediately visible to any subsequent read against the primary) and **eventual consistency** for propagation to connected clients: a change is always durably committed to MongoDB *before* any client is notified of it, but different clients may observe the notification at slightly different times depending on Change-Stream and network latency. This asymmetry is a deliberate, accepted trade-off: the domain (a SOC dashboard) tolerates sub-second staleness in what operators *see*, but never tolerates staleness or loss in what is actually *recorded* as the incident’s state.

3.7 Fault-Tolerance

Replication. MongoDB runs as a 3-node Replica Set. The loss of a secondary goes unnoticed; the loss of the primary costs no acknowledged write, since MongoDB 6 acknowledges a write only once a majority of the nodes holds it (its default write concern), but it suspends reads and writes until the two remaining nodes elect a new primary —

about 15s in the acceptance test (section 5.1), during which the `api` readiness probes may fail. Redis is deliberately *not* replicated (section 2.4, *Fault Tolerance, Dependability*): it holds coordination state, not the system of record, so its failure costs availability while it is down and the active claims, which no one reconstructs, but never durable data; the leadership is simply re-elected at the next tick. The 3–10 `api` pods and the 2 `web` pods are interchangeable workers, stateless with respect to domain state, rather than replicas of any data.

Heartbeats, timeouts, and retries.

- **Probes.** The liveness probe (`/api/health`) is shallow and the readiness probe (`/api/ready`) is deep: it checks MongoDB, Redis and the pod's own Change Stream. A failing dependency therefore takes a pod out of the Service without restarting it.
- **Leases.** Claims and leadership are Redis keys with a TTL: a crashed holder needs no failure detector, because its lock simply expires.
- **Connection heartbeat.** Socket.IO's ping/pong detects dead connections, which keeps the presence list computed with `io.fetchSockets()` correct.
- **Client retries.** The Socket.IO client reconnects automatically, and the resynchronization-generation counter (NFR7) discards out-of-date snapshots.
- **Change Stream recovery.** A broken stream is rebuilt with exponential backoff (1s doubling up to 30s) and `resumeAfter`, so the changes made during the outage are replayed. If the resume token is no longer in the oplog, the stream restarts from the present and the pod sends `resync_required` to its clients. If the stream is still down after 20s, the pod disconnects its clients so that they reconnect to a healthy pod.

Error handling. Four classes of failure are handled, each by the cheapest mechanism that recovers from it:

- **Backend crash.** An `api` pod that dies is restarted by Kubernetes, and one that hangs is restarted when its shallow liveness probe fails. Nothing is lost with it: claims and leadership live in Redis, and the leadership passes to another pod once the 15-second lease expires. Its WebSocket clients reconnect through the Ingress to another replica and rehydrate, while a REST request in flight fails and the client reports it as a network error. A failure to reach MongoDB or Redis *at startup*, instead, is fatal by design (`process.exit(1)`): the pod is restarted rather than serving traffic half-initialized.
- **Redis failure.** The Redis clients reconnect on their own, and the commands issued in the meantime wait in their offline queue. The readiness probe's PING therefore times out, every `api` pod leaves the Service without being restarted, and

the board stays unavailable until Redis returns; after a restart the active claims are lost, and the leadership is re-elected at the next tick (section 2.4).

- **MongoDB failure.** The loss of a secondary is invisible. The loss of the primary suspends reads and writes until the election: the driver waits for the new primary, retries a failed read or write once, and resumes the Change Streams from their tokens on its own; the errors it cannot resume are left to the pod's Change Stream recovery described above. If the whole Replica Set is down, readiness fails on every pod, as with Redis.
- **REST API errors.** Every REST handler catches its errors and answers with a specific status — 400/404 for invalid input, 401 for a missing or invalid token or wrong credentials, 409 for a lock, version or transition conflict, 500 otherwise — instead of crashing the process. On the frontend, every action checks the response and shows a notification on failure, distinguishing the server's message from a network error and tolerating bodies that are not JSON (e.g., a 502 from the Ingress); a token rejected at the WebSocket handshake sends the operator back to the login form.

3.8 Availability

Caching. No server-side cache is used for Incident data: every `GET /api/incidents` reads MongoDB directly. A cache would buy little: clients call it only at login, on reconnection and on `resync_required`, and live on events in between, so the read volume does not grow with the use of the board. The snapshot also carries each claim's remaining TTL, read from Redis at that instant, which a cache would serve stale. The role of a cache is played instead by each client, which keeps the whole board in memory and updates it from the event stream. Redis, despite superficially resembling a cache, is never used to cache MongoDB reads; its only roles are coordination (locks, leadership) and message brokering (the Socket.IO adapter).

Load balancing and sticky sessions. The Ingress load-balances HTTP and WebSocket traffic across the ≥ 3 `api` replicas with cookie-based **sticky sessions** (cookie `route`). Socket.IO requires them: every connection opens on HTTP long-polling and only then upgrades to WebSocket, and the handshake, the polling requests and the upgrade must all reach the *same* process, which holds the session. Affinity concerns this connection state only and does not contradict the stateless API tier: REST calls carry the cookie too, since the Ingress rule is shared, but any replica could serve them, because identity travels in the JWT. The cookie is a preference rather than a binding: a pod that fails or turns unready leaves the endpoint list, and the Ingress moves its clients to another replica. This is also why a pod whose Change Stream is down disconnects its clients only after 20 s (section 3.7): done earlier, the cookie would send them straight back to the same pod. The `HorizontalPodAutoscaler` keeps the pool of backends elastic under load (section 2.2).

Behaviour under network partitioning. In every scenario below the system favours **consistency** over the availability of the partitioned side:

- If an `api` pod loses connectivity to Redis or MongoDB, or its Change Stream cannot be re-established, `/api/ready` starts failing and the pod leaves the Service's endpoints without being killed, rejoining once connectivity returns; new requests reach only pods that can serve them correctly. Its open WebSockets are moved only when the Change Stream is what failed (after 20 s): a pod cut off from Redis alone keeps its clients, which miss the lock and presence events published meanwhile, a divergence bounded by the claims' 60-second TTL.
- If the leader is cut off from Redis, it can no longer renew its lease and stops escalating, and another pod takes over within 15 s. If it is cut off from MongoDB alone, it keeps the lease while its escalation scans fail, so escalation is suspended cluster-wide until the partition heals: no data is lost, since overdue tickets are escalated at the next successful scan.
- A MongoDB secondary partitioned from the primary falls behind and resynchronizes once the partition heals; clients do not notice, since every read goes to the primary.
- A primary partitioned from the majority steps down, so the minority side serves neither reads nor writes rather than risk divergent incident data (**CP**). The writes it accepted but could not replicate were never acknowledged, since acknowledgement requires a majority, and are rolled back when it rejoins.

3.9 Security

Authentication. Operators register with a username and a password of at least 8 characters; the password is stored only as a bcrypt hash, and the uniqueness of the username is enforced by a unique index rather than by an application-level check, which would race across replicas. Login answers with the same error for an unknown user and a wrong password, so it does not reveal which usernames exist, and issues a JWT (subject: the username, validity: 12 hours) signed with a secret shared by every `api` replica through a Kubernetes `Secret`. The token is required on every REST call (**Authorization: Bearer**) and on the Socket.IO handshake, and any pod can verify it locally: no session state is kept on the server. The browser keeps it in `sessionStorage`, so it survives a page reload but not the closing of the tab. The known limits are deliberate simplifications (section 2.4, *Security, Trust*): authentication is single-factor, there is no password reset, account lockout or rate limiting on login, and the WebSocket token is checked only at the handshake, so an open connection outlives the expiry of its token.

Authorization. There is a single role: every authenticated operator can create incidents and claim any unclaimed one. Releasing and resolving, instead, require *owning* the incident's Redis lock: the server answers 409 both when the lock belongs to someone

else and when there is no lock at all, so the mutual exclusion cannot be bypassed by simply not claiming (NFR2). The lock therefore acts as a capability for a single incident, and it is anchored to a verified identity: its value, like `createdBy` and `closedBy`, is always the username taken from the caller's JWT and never from the request body.

Input validation. Every route validates its input before touching the stores: incident titles must be non-empty and at most 200 characters, a status must belong to the allowed set and follow an allowed transition (a closed incident cannot be reopened), and the optimistic-locking `version` is mandatory. Incident identifiers are checked for format and existence before being turned into a Redis key, which prevents a client from creating arbitrary keys or announcing a lock on an incident that does not exist.

Cryptography and secrets. Cryptography is used only for authentication: bcrypt for passwords and HMAC-signed JWTs for tokens. Nothing travels encrypted: the Ingress serves plain HTTP, so passwords and tokens cross the network in clear, and the traffic between the `api` pods and MongoDB or Redis is unencrypted as well; neither store requires authentication, and no data is encrypted at rest. Moreover, the `Secret` committed in the repository holds a placeholder value meant for the local demo, which a real deployment must generate outside the repository. Transport and at-rest encryption are listed among the future works (section 9).

4 Implementation

4.1 Technology Stack

- **Frontend:** a single-page application in Vue 3 (Composition API, `<script setup>`), built with Vite. `socket.io-client` keeps the real-time channel, Chart.js (through `vue-chartjs`) draws the dashboard chart, and `vue-toastification` shows the notifications.
- **Backend:** Node.js 20 with Express 5 for the REST API and Socket.IO 4 for the WebSocket server, extended across pods by `@socket.io/redis-adapter`. Authentication uses `jsonwebtoken` and `bcryptjs`; the stores are reached through the official `mongodb` and `redis` (node-redis) drivers.
- **Database:** MongoDB 6 as a 3-member replica set, the durable store of incidents and users and the source of the Change Streams; Redis 7 as a single in-memory instance for the claim locks, the leader lease and the Socket.IO Pub/Sub.
- **Distributed infrastructure:** Kubernetes, validated on Minikube: Deployments for `api`, `web` and Redis, a StatefulSet with a headless Service for MongoDB, a one-shot Job for the replica-set initialization, an ingress-nginx Ingress with cookie affinity, a HorizontalPodAutoscaler fed by `metrics-server`, PodDisruptionBudgets and a Secret for the JWT key.

- **Containerisation:** Docker. The `api` image is based on `node:20-alpine` with production dependencies only; the `web` image is a multi-stage build that compiles the bundle on `node:20-alpine` and serves it with `nginx:alpine`. MongoDB and Redis use the official `mongo:6` and `redis:7` images.
- **API:** REST over HTTP with JSON bodies under `/api` (authentication, incidents, claims, health probes), authenticated with a **Bearer** JWT; server-to-client events over Socket.IO, on WebSocket with a long-polling fallback.

4.2 Network Protocols

Two application-level protocols coexist, each chosen for a specific interaction pattern:

HTTP/1.1 over TCP is used for every request from the client to the server: the **commands** (mutations) as well as the snapshot read used for hydration. The REST API exposed by Express.js defines nine endpoints:

- `POST /api/auth/register`, `POST /api/auth/login` — create an operator account, and exchange credentials for a JWT; the only client endpoints that do not require one;
- `POST /api/incidents` — create a new incident;
- `GET /api/incidents` — retrieve the full incident list, each document enriched with the current holder of its claim and that claim’s remaining time-to-live (used at connect time, on every reconnect, and on server-requested resynchronization);
- `POST /api/incidents/:id/claim` — acquire the distributed lock on an incident;
- `DELETE /api/incidents/:id/claim` — explicitly release a held lock;
- `PATCH /api/incidents/:id` — resolve an incident (with optimistic-locking version check and server-side lock verification);
- `GET /api/health`, `GET /api/ready` — the shallow liveness and deep readiness probes, called by the kubelet rather than by the client.

HTTP was chosen because every mutation requires an unambiguous, synchronous success-or-failure response (e.g., “did I get the lock or not?”), which request/response semantics provide naturally.

WebSocket (via Socket.IO) is used for all **event** (notification) interactions from the server to connected clients. The Socket.IO library negotiates a WebSocket upgrade after an initial HTTP handshake, falling back to HTTP long-polling if WebSocket is unavailable. The server emits five event types:

- `incident_update` — carries a MongoDB Change Stream document (insert or update) describing a persistent state change;

- `incident_locked` / `incident_unlocked` — notifies all clients of ephemeral lock state changes; `incident_locked` also carries the claim's time-to-live, so that each client can expire the lock locally when no event will announce its natural expiry;
- `users_update` — broadcasts the current list of connected operators;
- `resync_required` — asks the pod's own clients to re-hydrate from `GET /api/incidents`, emitted when the pod knows it has lost a segment of the change feed and cannot replay it (section 3.7).

WebSocket was chosen over polling because the design requires near-real-time propagation of every state change to every connected client.

Redis protocol (RESP). The `api` pods communicate with Redis using the native Redis Serialization Protocol over TCP, mediated by the `redis` npm package (v6). This includes standard commands (`SET`, `GET`, `DEL`, `PING`) and Lua script evaluation (`EVAL`) for atomic lock operations.

MongoDB Wire Protocol. The `api` pods communicate with the MongoDB Replica Set using the MongoDB binary wire protocol over TCP, mediated by the official `mongodb` npm driver (v7). The connection string enumerates all three replica-set members (`mongo-0.mongo:27017`, `mongo-1.mongo:27017`, `mongo-2.mongo:27017`) with `replicaSet=rs0`, enabling automatic primary discovery and failover-transparent reconnection.

4.3 In-Transit Data Representation

JSON is used as the sole serialization format for all inter-component communication that crosses a network boundary in the application layer:

- **REST API:** both request bodies (e.g., `{ "title": "...", "status": "open", "createdBy": "alice" }`) and response bodies are JSON. The Express.js middleware `express.json()` parses incoming bodies, and every route handler returns a JSON object via `res.json()`.
- **Socket.IO events:** each emitted event carries a JSON payload. For `incident_update`, the payload is the raw MongoDB Change Stream document (which is already a JSON-like BSON structure, automatically serialized to JSON by the driver). For `incident_locked` the payload is `{ id, lockedBy, ttl }` and for `incident_unlocked` simply `{ id }`; for presence, a plain array of username strings; `resync_required` carries no payload at all, being a pure signal. The `ttl` field is deliberately a *relative* duration in milliseconds rather than an absolute expiry timestamp: an absolute deadline would only be interpretable by a client whose clock agrees with the cluster's, and a few seconds of skew would make locks expire visibly too early or too late.

- **MongoDB ↔ driver:** the driver uses BSON (Binary JSON) on the wire, which is transparently serialized/deserialized by the `mongodb` npm package; the application code only ever manipulates plain JavaScript objects.
- **Redis ↔ driver:** Redis values are plain UTF-8 strings (usernames, pod identifiers); no structured serialization is needed because each key holds a single scalar value.

4.4 Database Querying

MongoDB is queried through the fluent API of the official Node.js driver.

Writes.

- `insertOne` creates an incident with `status: 'open', version: 1` and `updatedAt`, and registers a user; the uniqueness of usernames is enforced by a unique index.
- `updateOne({_id, version, status: {$in: allowed sources}}, {$set: ..., $inc: {version: 1}})` implements optimistic locking: the update applies only if nobody has changed the document since the client read it and the transition is allowed from its current status.
- `updateMany({status: 'open', updatedAt: {$lt: threshold}}, ...)` escalates every overdue incident in a single round-trip.

Reads. The board snapshot is `find({}).sort({updatedAt: -1})`, enriched with the holder and remaining TTL of each claim read from Redis, so that every client hydrates to the same state whenever it connects; documents are read before locks, so a lock acquired in between is reported rather than lost. The other reads are point lookups: the user at login, the existence of an incident before any claim, and its current status and version after a failed update, to tell a version conflict from a forbidden transition.

Change Streams. `watch([], {fullDocument: 'updateLookup'})` streams every insert and update with the full document, replacing any polling. Each event's `_id` is kept as a resume token and passed as `resumeAfter` when the cursor is rebuilt; the driver already retries resumable errors such as a primary failover, so the application only handles the rest (section 3.7).

Redis. Single-key commands (`SET NX PX`, `GET`) plus three Lua scripts, which Redis executes atomically:

- `RENEW_LOCK_SCRIPT` extends a key's TTL only if it still belongs to the caller; it renews both the leadership and the claim of its current holder.
- `RELEASE_LOCK_SCRIPT` deletes a claim only if it still belongs to the caller, so an expired lock taken over by someone else is never deleted by mistake.

- `READ_LOCKS_SCRIPT` returns owner and remaining TTL of every existing lock in one call: the two values refer to the same instant, and hydration costs one round-trip regardless of the size of the board.

4.5 Technological Details

4.5.1 Backend Runtime and Frameworks

- **Node.js 20** (Alpine-based Docker image, `node:20-alpine`): chosen as the application runtime because its single-threaded, event-loop-based execution model is well suited to hosting both synchronous request handlers and long-lived WebSocket connections in the same process, without the thread-management overhead of a multi-threaded server.
- **Express.js 5**: provides the HTTP routing layer. An `express.Router()` instance mounts all endpoints under the `/api` prefix; the `express.json()` middleware handles request body parsing; and `cors()` is configured with the `CORS_ORIGIN` environment variable to restrict cross-origin requests in deployment.
- **Socket.IO 4**: provides the WebSocket transport layer, with automatic fallback to HTTP long-polling if WebSocket upgrade fails. The `@socket.io/redis-adapter` (v8) bridges all Socket.IO instances across the `api` pods so that events emitted with `io.emit()` are propagated cluster-wide via Redis Pub/Sub.
- **mongodb 7** (official Node.js driver): provides the connection to the MongoDB Replica Set, including Change Streams support and automatic primary failover handling.
- **redis 6** (Node.js client): provides the connection to Redis for distributed locks, leader election, and the Socket.IO adapter's Pub/Sub channels. Two client instances are created (`pubClient` and `subClient`), as required by the Redis adapter which needs dedicated publish and subscribe connections.

4.5.2 Frontend Framework and Libraries

- **Vue.js 3** with the Composition API (`<script setup>`): the SPA framework. The entire dashboard is a single `App.vue` component using Vue's reactivity system (`ref`, `computed`) to derive filtered views of the incident list and KPI counters from a single reactive `incidents` array that is kept current by the Socket.IO event stream.
- **Vite 8**: the build tool and development server. The production build (triggered by `npm run build` inside the multi-stage Dockerfile) produces a static bundle served by Nginx.

- **socket.io-client 4**: the browser-side Socket.IO library. The connection is established with `io('/', {path: '/socket.io', auth: {token}})`, which leverages the Ingress's path-based routing to reach the `api` service; the token is the same JWT used for REST calls, verified by the server on the handshake.
- **vue-toastification 2**: provides non-blocking toast notifications for all real-time events (new tickets, escalations, resolutions by others) and for local operation failures (claim conflicts, version conflicts, network errors). Configured in `main.js` with bottom-right positioning, 5-second timeout, and a progress bar.
- **Chart.js 4** (via `vue-chartjs 5`): renders the bar chart in the “Incidents Status Overview” card, with reactive data bindings to the computed KPI properties so the chart updates automatically as the incident counts change.

4.5.3 Containerization

Both application components use **Docker** multi-stage or single-stage builds:

- **API image**: a single-stage build from `node:20-alpine`. The `Dockerfile` copies `package*.json` first and runs `npm ci --only=production` to leverage Docker's layer cache, then copies the application code. The entrypoint is `node index.js`.
- **Web image**: a multi-stage build. Stage 1 (`node:20-alpine`) installs dependencies and runs `npm run build` to compile the Vue SPA into static assets. Stage 2 copies the `dist/` output into an `nginx:alpine` image, producing a minimal container that serves only static files on port 80.
- **MongoDB and Redis**: use the official upstream images (`mongo:6` and `redis:7`) without custom Dockerfiles.

Both custom images set `imagePullPolicy: Never` in their Kubernetes manifests, instructing Kubernetes to use the locally-built images available in Minikube's Docker daemon rather than attempting to pull them from a remote registry.

4.5.4 Orchestration (Kubernetes)

The Kubernetes manifests in `k8s/` define the following resources, all deployed in the `app` namespace:

- **API Deployment + Service (api.yaml)**: a `Deployment` with 3 initial replicas, each running the `api:latest` image. Environment variables configure the MongoDB connection string (enumerating all three replica-set members), the Redis URL, and the CORS origin. The liveness probe (`/api/health`, shallow) and readiness probe (`/api/ready`, deep) are deliberately separated: a dependency outage only removes the pod from the Service's endpoint list (readiness failure) without restarting a healthy process (liveness would pass).

- **HorizontalPodAutoscaler** (`hpa.yaml`): targets the `api` Deployment, scaling between 3 and 10 replicas based on an 80% average CPU utilization threshold.
- **MongoDB StatefulSet + Headless Service + ConfigMap + Job + PDB** (`mongo.yaml`): the most complex manifest, comprising five resources:
 - A **headless Service** (`clusterIP: None`) that provides stable per-pod DNS names (`mongo-0.mongo`, `mongo-1.mongo`, `mongo-2.mongo`), necessary because replica-set members must be individually addressable.
 - A **ConfigMap** containing the `rs.initiate()` JavaScript invocation with the three member hostnames.
 - A **StatefulSet** with 3 replicas, each running `mongo:6` with `--replSet rs0 --bind_ip_all`. Each pod has a `volumeClaimTemplate` requesting 1 GiB of persistent storage, ensuring data survives pod restarts. Liveness and readiness probes execute `db.adminCommand('ping')` via `mongosh`.
 - A one-shot **Job** (`mongo-rs-init`) that waits for all three members to be reachable, checks whether the replica set is already initialized, and if not, runs the `rs.initiate()` script and waits for a primary to be elected. The `backoffLimit: 5` and `restartPolicy: OnFailure` ensure the initialization is retried if it fails due to a transient readiness issue.
 - A **PodDisruptionBudget** (`minAvailable: 2`) that prevents voluntary evictions (e.g., node drains) from disrupting more than one MongoDB pod at a time, preserving the quorum needed for automatic primary election.
- **Redis Deployment + Service** (`redis.yaml`): a single-replica Deployment running `redis:7`, exposed via a `ClusterIP` Service on port 6379. No persistence, no replication — a deliberate simplification documented in section 2.4.
- **Web Deployment + Service** (`web.yaml`): a single-replica Deployment running the Nginx-based `web:latest` image, exposed via a `ClusterIP` Service on port 80.
- **Ingress** (`ingress.yaml`): an Nginx Ingress resource configured for host `board.local` with three path-based routing rules:
 - `/` → `web:80` (static frontend assets);
 - `/api` → `api:3000` (REST endpoints);
 - `/socket.io` → `api:3000` (WebSocket transport).

Cookie-based sticky sessions are enabled via annotations (`nginx.ingress.kubernetes.io/affinity: cookie`, cookie name `route`, 48-hour expiry) because every Socket.IO connection starts on HTTP long-polling before upgrading to WebSocket, and all of its requests must reach the same backend pod.

5 Validation

5.1 Acceptance Tests

All validation was performed by **manual testing** on the live Minikube cluster, with several browser sessions and `curl`. Tests are grouped by mechanism rather than by requirement, and each lists the requirements it covers. Unless stated otherwise, the results were observed on the cluster; steps marked (*expected*) cover behaviour introduced after the last validation round and describe what the code is meant to do, not an observed result.

Automating these scenarios would require orchestrating several browser sessions, injecting pod failures through the Kubernetes API, and waiting for infrastructure reactions such as probe recovery or HPA decisions. Playwright or Cypress could cover the browser side, but integrating them with `kubect1`-driven fault injection was beyond the time budget of a single-developer project.

In the commands below, `ALICE_TOKEN` and `BOB_TOKEN` are JWTs obtained from `POST /api/auth/login`. The identity behind every write (creator, lock owner, resolver) is always taken from the token, never from the request body.

5.1.1 Real-Time Synchronization (FR2, FR3, NFR9)

- **Procedure:** log in as “alice” and “bob” in two browsers. As alice, create an incident, claim it and resolve it, observing bob’s board after each step.
- **Expected:** the creation answers 201, and every step appears on bob’s board within about one second, with no refresh. At no point does the card appear in two columns at once, whichever of the lock-release and database events reaches the client first.
- **Result:** passed. Creation and resolution travel through the Change Stream (`incident_update`, emitted by each pod to its own clients), the claim through the Redis adapter (`incident_locked`). The columns are derived from both `status` and `lockedBy`, so any intermediate state falls into exactly one of them.

5.1.2 Claim and Release (FR4, FR5)

- **Procedure:** claim the same incident concurrently as alice and bob:

```
curl -X POST http://board.local/api/incidents/<id>/claim \  
-H "Authorization: Bearer $ALICE_TOKEN" &  
curl -X POST http://board.local/api/incidents/<id>/claim \  
-H "Authorization: Bearer $BOB_TOKEN" &  
wait
```

Then release the claim as its winner (`DELETE /api/incidents/<id>/claim`) and claim it again as the loser.

- **Expected:** exactly one claim answers 200 (`{"success":true, "lockedBy":"...", "ttl":60000}`), the other 409 naming the owner. The release answers 200, the card returns to its column on every client, and the second claim succeeds.
- **Result:** passed. `SET NX PX` is atomic, and `RELEASE_LOCK_SCRIPT` deletes the key only if it still belongs to the caller.
- (*Expected*) Claiming again as the current holder answers 200 rather than 409, with the remaining TTL (`redis-cli PTTL lock:incident:<id>`) back at about 60000 ms; the same request from the other user still answers 409.

5.1.3 Server-Enforced Mutual Exclusion and Resolution (NFR2, FR6)

- **Procedure:** claim an incident as alice, then try to resolve it as bob, bypassing the UI:

```
curl -X PATCH http://board.local/api/incidents/<id> \
-H "Authorization: Bearer $BOB_TOKEN" \
-H "Content-Type: application/json" \
-d '{"status":"closed","version":1}'
```

Finally, resolve it as alice from the UI.

- **Expected:** bob's request is rejected with 409 ("Incidente bloccato da un altro utente", `lockedBy: "alice"`), although bob uses his own genuine credentials. Alice's resolution answers 200: the card moves to "Resolved" on every client with alice as resolver, and offers no further action.
- **Result:** passed. The server compares the lock owner with the identity in the verified JWT, and releases the lock atomically within the same request.
- (*Expected*) A PATCH on an incident that nobody has claimed is rejected with 409 as well ("Devi prendere in carico l'incidente prima di modificarlo"), so skipping the claim does not bypass the mutex.

5.1.4 No Lost Updates (NFR1)

- **Procedure** (*expected*): create an incident and claim it as alice when it is about four and a half minutes old, so that the claim is still held when the SLA task escalates it (`version 1` $\rightarrow$ `2`). Then send, as alice, `{"status":"closed","version":1}`.
- **Expected:** the request is rejected with 409 ("Conflitto! Il documento è stato già modificato o non esiste.") and the document is left unchanged (`escalated`, `version 2`); repeating it with `version:2` succeeds. This is the real race between an operator and the leader's background write.

- **Note:** this procedure replaces an earlier one, two consecutive resolutions with the same version, which passed before the server required lock ownership. It no longer exercises optimistic locking, because the second request is now rejected by the lock check before the version is compared; the new procedure has not yet been reproduced on the cluster.

5.1.5 SLA Escalation, Exactly Once (FR7, NFR5)

- **Procedure:** with ≥ 3 `api` pods running, leave an incident `open` for more than 5 minutes, watching the boards and the combined logs (`kubectl logs -l app=api -n app | grep Escalati`).
- **Expected:** within 10 seconds of the threshold the incident becomes `escalated`, moves to “Escalated” on every client with a warning toast, and its `version` grows by exactly 1. Exactly one log line reports it, from a single pod.
- **Result:** passed. Only the holder of `leader:escalation` runs the task, and the Change Stream propagates the result.

5.1.6 Presence (FR8, NFR8)

- **Procedure:** connect alice and bob and check that “Users Online” shows 2; close bob’s tab. Reconnect bob, then force-terminate the pod serving him (`kubectl delete pod <pod> -n app --force --grace-period=0`), so that no `disconnect` event is ever emitted.
- **Expected:** the counter drops to 1 when the tab is closed and, after the forced termination, converges to the correct value (1, or 2 once bob has reconnected elsewhere) without any manual cleanup.
- **Result:** passed. Presence is computed by `io.fetchSockets()` across the adapter, and dead connections are pruned by the Socket.IO heartbeat. The earlier implementation, a Redis hash maintained by hand, failed this test with entries that never expired, which motivated the switch.

5.1.7 API Pod Failure (NFR3)

- **Procedure:** delete one of the ≥ 3 `api` pods (`kubectl delete pod <name> -n app`) and immediately create and claim an incident from the browser.
- **Expected:** no visible downtime; the Deployment recreates the pod, and clients that were connected to it reconnect to a surviving pod.
- **Result:** passed. The pod was recreated within seconds, and the client’s `connect` handler re-fetched the board.

- **Change Stream as a readiness dependency** (*expected*). A pod whose Change Stream has died answers 503 on `/api/ready` (“change stream non attivo”) and leaves the Service without being restarted, since the liveness probe is shallow; its clients are moved to a ready pod, and it rejoins once the cursor is rebuilt (log lines `Change stream interrotto` and `Change stream ripristinato`). A non-resumable stream failure is hard to inject from outside the process — the driver silently retries the resumable ones, so deleting a `mongo` pod does not exercise this path — and this check has not been reproduced on the cluster.

5.1.8 Data-Tier Failover (NFR4)

- **Procedure:** find the MongoDB primary (`kubectl exec mongo-0 -n app -- mongosh --quiet --eval "rs.isMaster().primary"`), delete its pod, and follow `rs.status()` from a secondary.
- **Expected:** the two remaining members elect a new primary within 10–30 seconds. The `api` readiness probes may fail briefly, taking the pods out of the Service, and pass again once the primary is available. The deleted pod is recreated by the StatefulSet and rejoins as a secondary.
- **Result:** passed, with the election completed in about 15 seconds. Deleting a pod directly is not an eviction, so the PodDisruptionBudget plays no part here: it guards quorum against voluntary disruptions such as a node drain.

5.1.9 Reconnection Consistency (NFR7)

- **Procedure:** with a client connected, run `kubectl rollout restart deployment/api -n app`.
- **Expected:** the client reconnects on its own and always shows the current board, never a snapshot fetched by a superseded connection.
- **Result:** passed. The client went through 1–2 brief disconnections and always converged to the correct state; the resynchronization-generation counter discards stale responses.
- (*Expected*) Claim an incident as alice and reload her tab: the card is still “In Progress” with “Locked by: alice” and her **Resolve/Release** buttons, because the snapshot carries `lockedBy` and a `lockTtl` below 60000. Left idle, the card returns to its column on every client when the TTL elapses, with no server event.

5.1.10 Horizontal Scalability (NFR6)

- **Procedure:** generate sustained CPU load on the `api` pods (e.g., a tight `curl` loop) and watch `kubectl get hpa api-hpa -n app -w`.

- **Expected:** above 80% average CPU the HPA adds replicas, up to 10, within its stabilization window.
- **Result:** passed: scale-up from 3 to 5 replicas under artificial load.

5.1.11 Client-Side Checks (FR1, FR9, FR10)

These requirements involve no distributed mechanism and were checked directly in the browser:

- **FR1** — registration and login lead to the dashboard with an authenticated Web-Socket (JWT in the handshake's `auth.token`); wrong credentials, short passwords and duplicate usernames are rejected by the server regardless of the client. *Passed*.
- **FR9** — search and the two toggles change only the Kanban columns, while the KPI counters, computed from the unfiltered list, stay unchanged. *Passed*.
- **FR10** — creations and resolutions by others, escalations and every failed action produce a toast; failures carry the server's reason. *Passed* for claim, release and resolve. Incident creation, which showed a success toast even when the request failed, now reports failures as the other actions do (*expected*).

6 Deployment

6.1 Prerequisites

The following software must be installed before proceeding. Where a version is given it is a minimum imposed by the project's dependencies; the procedure was developed on Docker 29.8, Minikube 1.39, kubectl 1.37 and Node.js 22.13.

- **Docker Desktop** (or Docker Engine) — the runtime on which Minikube's `docker` driver runs the cluster node. The images themselves are built inside Minikube's own Docker daemon (section 6.3); the host only provides the `docker` client.
- **Minikube** — provides the single-node Kubernetes cluster, with the `ingress` and `metrics-server` addons. The `docker` driver is assumed throughout.
- **kubectl** — within one minor version of the Kubernetes release that Minikube provisions, as required by the Kubernetes version-skew policy. `minikube kubectl --` downloads a matching client and can be used in its place.
- **Node.js** 20.19+ or 22.12+, with the bundled `npm` — only needed to run the API or the frontend outside of containers; the Kubernetes deployment builds both inside the images. The bounds come from the MongoDB driver 7 (`>=20.19.0`) and from Vite 8 (`~20.19.0 || >=22.12.0`).

Cluster resources. The containers request $3 \times 100\text{m}$ CPU for `api`, $3 \times 100\text{m}$ for `mongo` and $2 \times 10\text{m}$ for `web`, on top of roughly one core taken by the control plane, the Ingress controller and `metrics-server`. This fits Minikube's default of 2 CPUs at the initial replica count, but not once the HPA scales `api` towards its maximum of 10 replicas: the pods beyond the node's capacity remain `Pending`. The cluster is therefore started with 4 CPUs and 4 GB of memory (Step 1 below), which Docker Desktop must be allowed to allocate.

6.2 Cluster Setup

Step 1: Start Minikube. Create a local cluster with the Ingress addon enabled:

```
minikube start --driver=docker --cpus=4 --memory=4g
minikube addons enable ingress
minikube addons enable metrics-server    # required for the HPA
```

The `ingress` addon deploys an Nginx Ingress Controller inside the cluster. The `metrics-server` addon is necessary for the `HorizontalPodAutoscaler` to read CPU utilization metrics.

Step 2: Create the namespace. All project resources are deployed in the `app` namespace:

```
kubectl create namespace app
```

Step 3: Make `board.local` reachable. The Ingress routes traffic based on the host-name `board.local`, so the host must resolve that name to an address where the Ingress controller listens. Which address that depends on the platform.

Linux. The Minikube node's IP is routable from the host. Add the following entry to `/etc/hosts` (requires administrator privileges), where `<minikube-ip>` is the output of `minikube ip`:

```
<minikube-ip> board.local
```

macOS with the `docker` driver. The node runs inside Docker Desktop's virtual machine and its IP is not routable from the host. `minikube tunnel` does not help either: it exposes only Services of type `LoadBalancer`, while the Ingress controller installed by the addon is a `NodePort` Service. The controller is instead reached through a port-forward, kept running in a separate terminal:

```
kubectl port-forward -n ingress-nginx \
    svc/ingress-nginx-controller 18080:80
```

and the name is mapped to the loopback address in `/etc/hosts`:

```
127.0.0.1 board.local
```

The browser then sends `Host: board.local`, which is what the Ingress rule matches. On this platform every URL in the remainder of the report takes the port explicitly: `http://board.local:18080` in place of `http://board.local`.

6.3 Building the Container Images

Because the Kubernetes manifests set `imagePullPolicy: Never`, the images must be built *inside* Minikube's Docker daemon, not the host's. This is achieved by pointing the local shell's Docker client at Minikube's daemon:

```
eval $(minikube docker-env)
```

Then, from the project root:

```
docker build -t api:latest ./api
docker build -t web:latest ./web
```

Expected outcome: two images (`api:latest`, `web:latest`) are available inside Minikube's Docker daemon. They can be verified with `docker images | grep -E 'api|web'`.

API image (`api/Dockerfile`): a single-stage build from `node:20-alpine`. It copies `package*.json` first, runs `npm ci --only=production` to install dependencies (leveraging Docker's layer cache for faster rebuilds), then copies the application code. The entrypoint is `node index.js`, listening on port 3000.

Web image (`web/Dockerfile`): a multi-stage build. Stage 1 (`node:20-alpine`) installs all dependencies (including dev dependencies needed by Vite) and runs `npm run build`, producing the compiled Vue 3 SPA in `/app/dist`. Stage 2 copies only the `dist/` directory into an `nginx:alpine` image, yielding a minimal production image that serves static files on port 80 with no Node.js runtime.

Third-party images. MongoDB (`mongo:6`) and Redis (`redis:7`) use the official upstream images directly and are pulled automatically by Kubernetes on first deployment (they do *not* have `imagePullPolicy: Never`).

6.4 Deploying the Kubernetes Manifests

The manifests in `k8s/` must be applied in a specific order to respect inter-component dependencies. Each step includes the command, the resources it creates, and the expected outcome.

Step 1: MongoDB (StatefulSet, Headless Service, ConfigMap, Init Job, PDB).

```
kubectl apply -f k8s/mongo.yaml
```

This creates five resources: the headless Service (`mongo`), the ConfigMap (`mongo-init`) containing the `rs.initiate()` script, the StatefulSet with 3 replicas, the initialization Job (`mongo-rs-init`), and the PodDisruptionBudget (`mongo-pdb`).

Expected outcome: the three `mongo` pods start in order (`mongo-0`, `mongo-1`, `mongo-2`); the init Job waits for all three to become reachable, then initializes the Replica Set `rs0` and waits for a primary election. This step takes approximately 60–90 seconds. Progress can be monitored with:

```
kubectl get pods -n app -l app=mongo -w
kubectl logs job/mongo-rs-init -n app -f
```

Step 2: Redis (Deployment, Service).

```
kubectl apply -f k8s/redis.yaml
```

Expected outcome: a single redis pod starts and becomes ready within a few seconds.

Step 3: JWT Secret. The api Deployment requires the `jwt-secret` Secret to exist before it starts, since `JWT_SECRET` is injected via `secretKeyRef`:

```
kubectl apply -f k8s/jwt-secret.yaml
```

Expected outcome: the Secret `jwt-secret` is created in the `app` namespace, holding the HMAC key shared by all `api` replicas. For any deployment beyond local development/demo purposes, the placeholder value committed in the manifest should be replaced, e.g. by regenerating it with `kubectl create secret generic jwt-secret --namespace app --from-literal=JWT_SECRET=$(openssl rand -base64 48) --dry-run=client -o yaml | kubectl apply -f -`.

Step 4: API (Deployment, Service).

```
kubectl apply -f k8s/api.yaml
```

Expected outcome: 3 `api` pods start. Each pod connects to the MongoDB Replica Set and to Redis on startup, and reads the shared `JWT_SECRET` from the Secret created in Step 3; if any dependency is not yet available, the pod exits with a fatal error and Kubernetes restarts it (the readiness probe prevents it from receiving traffic until all dependencies are reachable). Steady state is reached within 20–30 seconds:

```
kubectl get pods -n app -l app=api
```

All 3 pods should show `Running` with `READY 1/1`.

Step 5: HorizontalPodAutoscaler.

```
kubectl apply -f k8s/hpa.yaml
```

Expected outcome: the HPA `api-hpa` is created, targeting the `api` Deployment with a 3–10 replica range and an 80% CPU utilization target. The `metrics-server` addon must be running for the HPA to function:

```
kubectl get hpa -n app
```

Step 6: Web frontend (Deployment, Service).

```
kubectl apply -f k8s/web.yaml
```

Expected outcome: a single `web` pod starts, running the Nginx container that serves the compiled Vue 3 SPA on port 80.

Step 7: Ingress.

```
kubectl apply -f k8s/ingress.yaml
```

Expected outcome: the Ingress resource `app-ingress` is created with three path-based routing rules (`/` → `web:80`, `/api` → `api:3000`, `/socket.io` → `api:3000`) and cookie-based sticky sessions enabled. After a few seconds, `kubectl get ingress -n app` should show an `ADDRESS` matching the Minikube IP.

Verification. Once all manifests are applied, the full deployment can be verified in a single command:

```
kubectl get all -n app
```

Expected output should show: 3 `api` pods (Running), 3 `mongo` pods (Running), 1 `redis` pod (Running), 2 `web` pods (Running), 1 completed Job (`mongo-rs-init`), plus their respective Services, Deployments, StatefulSet, and ReplicaSets.

Opening `http://board.local` in a browser should display the login screen (`http://board.local:18080` on macOS, with the port-forward of section 6.2, Step 3, running).

7 User Guide

This section describes the application from the operator’s point of view, assuming the deployment of section 6 is reachable at `http://board.local` (`http://board.local:18080` on macOS). The interface labels are in Italian and English, and are quoted here as they appear on screen.

7.1 Accessing the Board

A first-time operator clicks “Registrati” on the login card, chooses a username and a password of at least 8 characters, and is brought back to the login form. Logging in with those credentials (“Accedi”) opens the dashboard and establishes the real-time connection in the background; wrong credentials are rejected with an inline error. The session survives a page reload and ends when the tab is closed, when the operator clicks “Esci” in the header, or after 12 hours.

7.2 Dashboard Overview

The dashboard is organized in four rows:

1. **KPI cards:** Total, Open, Escalated and Resolved tickets (by status, whether claimed or not) and Users Online, the number of distinct operators currently connected.
2. **Status chart and creation form** (“Register New Incident”).
3. **Filters:** a free-text search on the title (case-insensitive), “Mostra solo Escalated”, and “I Miei Ticket”, which keeps the incidents the operator created, holds, or resolved. Filters can be combined and affect *only* the Kanban columns: the KPI cards always describe the whole board.
4. **Kanban board** with four columns, each with a counter of its cards:
 - **Open** — unclaimed incidents with status **open**, with a “Claim Ticket” button;
 - **In Progress** — every claimed incident, whatever its status (an “ESCALATED” badge marks escalated ones), showing “Locked by: <name>”; only the holder sees the “Resolve” and “Release” buttons;
 - **Escalated** — unclaimed incidents past the SLA, with a “Claim Escalation” button;
 - **Resolved** — closed incidents, showing “Solved by: <name>”, with no actions.

The “Live Sync Active” indicator in the header shows that the real-time connection is up.

7.3 Incident Lifecycle

Create. Type a title in “Register New Incident” and press **Enter** or “Submit”. The card appears in “Open” on every connected client, without any refresh.

Claim. Click “Claim Ticket” (or “Claim Escalation”). The card moves to “In Progress” on every client, and only the claimant gets the action buttons. If another operator got there first, an error toast names the current holder and nothing changes. A claim lasts **60 seconds**: if it is neither resolved nor released by then, the card returns to its column on every client, including the holder’s, and anyone can claim it again. Reloading the page does not lose a claim that is still valid.

Release. Click “Release” to give the incident back without resolving it. The card returns to “Open” or “Escalated” and is immediately claimable by others.

Resolve. Click “Resolve”. The card moves to “Resolved” on every client, with the operator’s name as resolver, and can no longer be claimed or modified. Only the current holder can resolve, and the server enforces this even for requests that bypass the interface. If the incident was escalated in the instant between the moment the card was last updated and the click, the request is rejected as a conflict (“Conflitto o errore: ...”); retrying once the card shows the new status succeeds, since the claim is still held.

Automatic escalation. An incident still **open** after **5 minutes** is escalated by the server, with no operator action: an unclaimed card moves to “Escalated”, a claimed one stays in “In Progress” and gains the “ESCALATED” badge, and every client shows a warning toast. Escalated incidents follow the same claim/release/resolve workflow.

7.4 Notifications

Non-blocking toasts appear in the bottom-right corner and disappear after 5 seconds (8 for escalations); they can be dismissed by clicking or dragging them. They report the operator’s own successful actions, incidents created or resolved by others (info and success), escalations (warning), and every failed action with the server’s reason, such as a claim conflict or a network error (error).

7.5 Collaboration and Connection Resilience

The board is meant to be used by several operators at once: opening it in two browser windows with different users is enough to see cards move on both as either acts, the “Users Online” counter change as windows are opened and closed, and a claim on an incident held by the other user rejected immediately.

Connection problems require no action from the operator. If the connection drops (network glitch, pod restart, rolling update), the client reconnects by itself and reloads the whole board from the server, claims included, so it never shows a stale or partial state; the server can also ask its clients to reload when it detects that it has missed updates. Nothing is lost in the meantime: every action travels over HTTP and is either stored or reported as failed, while the real-time connection is used only to receive updates.

8 Self-evaluation

8.1 Role

The project was carried out individually, so every part of it is my own work: the elicitation of the requirements, the design of the architecture and of the coordination mechanisms, the implementation of the **api** backend and of the **web** frontend, the Kubernetes manifests, the manual validation on the Minikube cluster, and this report. Without a team there was also no reviewer: the defects recounted in section 2.4 were found by re-reading my own code against the properties it was meant to guarantee.

8.2 Strengths

- **One coordination primitive per problem.** Exclusive handling of a ticket, concurrent writes and the background task are three different problems, solved with three different tools — a Redis lock with atomic Lua scripts, optimistic locking on `version`, and leader election — each enforced by the server rather than trusted to the interface.
- **Real-time propagation without duplicates or polling.** Each event travels on exactly one channel: database changes through the per-pod Change Stream with `io.local.emit`, lock and presence events through the Redis adapter. Every client receives each event once, whichever pod it is connected to.
- **Recovery is designed, not assumed.** The Change Stream resumes from its token, its health drives the readiness probe, clients of a degraded pod are moved elsewhere, and every reconnection re-hydrates the whole board, claims included.
- **A genuinely stateless API tier.** Identity travels in a JWT and coordination state lives in Redis and MongoDB, so any request can land on any pod and the tier scales from 3 to 10 replicas without session migration.
- **Redundancy wherever it is cheap.** A 3-node MongoDB Replica Set, 3+ `api` and 2 `web` replicas, PodDisruptionBudgets, shallow/deep probes and `preStop` hooks keep the board available through pod crashes, rolling updates and the loss of a database node.

8.3 Weaknesses

- **Redis is a single point of failure.** It is the only component without redundancy, and its loss makes the whole board unavailable rather than read-only; having no persistence, a restart also drops every active claim (section 9).
- **The locks are not fencing-safe.** Without a fencing token, a leader paused beyond the 15-second TTL can overlap with its successor. It is harmless here only because escalation is idempotent, not because the mechanism prevents it.
- **Validation is manual.** There is no automated test suite, so every regression check means re-running the scenarios by hand, and the checks added after the last validation round — the Change Stream as a readiness dependency among them — are documented as expected behaviour rather than observed (section 5.1).
- **Availability is pod-level, not node-level.** Minikube runs a single node: the replicas protect against crashes and rollouts, not against the loss of the machine.
- **Minimal security.** No TLS between components, a single role, no rate limiting on login, and a WebSocket session that outlives the expiry of its token (section 3.9).
- **Scalability limits.** Each `api` pod holds its own Change Stream cursor on the primary, and every reconnection downloads the whole, unpaginated board.

9 Future works

The current iteration deliberately scopes out several directions in order to keep the focus on the distributed-systems aspects of the project. The most relevant candidates for future work are:

- **Transport security.** Traffic between the browser and the Ingress, and between the `api` pods and their backing services (MongoDB, Redis), is currently unencrypted (section 2.4). A production-grade deployment would terminate TLS at the Ingress and enable TLS for the MongoDB Replica Set and Redis connections, plus encryption at rest for the MongoDB volumes.
- **Fine-grained authorization.** All authenticated operators currently share a single implicit role. Introducing roles (e.g., operator vs. supervisor) with differentiated permissions — such as restricting who can resolve escalated incidents, or exposing an audit trail of who claimed/released/resolved which incident — would be a natural extension of the authorization model.
- **Account lifecycle hardening.** Login currently has no rate limiting, account lockout, or password-reset flow. Adding these (e.g., exponential backoff per username/IP, a lockout after repeated failures) would close an obvious brute-force gap without changing the stateless nature of the `api` tier.
- **Responsive / mobile layout.** The dashboard currently uses a fixed-column CSS Grid layout designed for a desktop viewport, with no `@media` breakpoints. Supporting narrower viewports would require reflowing the KPI row and Kanban columns (e.g., collapsing to a single-column, tab-based view below a given width) rather than a purely cosmetic change.
- **Drag-and-drop interactions.** Kanban cards currently move between columns automatically as a side effect of status/lock changes; there is no direct manipulation. Adding drag-and-drop (e.g., dragging a card into “In Progress” to claim it) would need to reconcile the optimistic client-side move with the same claim/release/resolve endpoints and lock semantics already in place, rather than introduce a separate mutation path.
- **Horizontal scaling of stateful services.** MongoDB already runs as a 3-node Replica Set and `api` scales horizontally via the HPA, but Redis remains a single instance (section 3.7) and is therefore both a scalability ceiling and the one component whose loss makes the whole system unavailable. The degradation is not partial: every `api` readiness probe pings Redis, so while Redis is down *all* replicas fail readiness at once and leave the Service together, and the board stops serving reads as well as writes. No durable data is at risk — incidents and credentials live in MongoDB — but the coordination state is: Redis runs without persistence, so a restart silently drops every active claim and the current leadership, and operators must claim their tickets again. Moving to Redis Sentinel or Redis Cluster would

remove this single point of failure at the cost of additional operational complexity. A cheaper intermediate step would be a degraded read-only mode, which does *not* exist today: it would require both dropping Redis from the readiness probe and letting `GET /api/incidents` return the snapshot without lock information when `READ_LOCKS_SCRIPT` cannot run, while claims, resolutions and cross-pod broadcasts remain unavailable.